

CA-1 (Source Code)

Name: Shivang Chaurasia

Roll No: 25

Question 1:

Source Code:

```
import java.util.*;
import java.util.stream.Collectors;

class Student {
    String name;
    String stream;
    String department;
    double cgpa;

    public Student(String name, String stream, String department,
double cgpa) {
        this.name = name;
        this.stream = stream;
        this.department = department;
        this.cgpa = cgpa;
    }

    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }

    public double getCgpa() {
        return cgpa;
    }

    @Override
    public String toString() {
        return name + " (" + cgpa + ")";
    }
}

public class Q1_Streams {
    public static void main(String[] args) {
        List<Student> students = Arrays.asList(
            new Student("Aman", "BTech", "CSE", 8.5),
            new Student("Aditya", "BTech", "IT", 7.5),
            new Student("Chandan", "BTech", "CSE", 9.0),
            new Student("Shivang", "BTech", "ECE", 6.8),
```

```

        new Student("Uday", "BTech", "ME", 8.2));

    Map<Boolean, List<Student>> reportA = students.stream()
        .collect(Collectors.partitioningBy(s -> s.getCgpa()
> 8.0));

    System.out.println("--- Report A: CGPA > 8.0 ---");
    System.out.println("Score > 8.0: " + reportA.get(true));
    System.out.println("Score <= 8.0: " + reportA.get(false));

    Map<String, List<Student>> reportB = students.stream()
        .collect(Collectors.groupingBy(s ->
s.getDepartment()));

    System.out.println("\n--- Report B: Group by Department ---
");
    reportB.forEach((dept, list) -> System.out.println(dept +
": " + list));

    Map<String, List<String>> reportBNames = students.stream()
        .collect(Collectors.groupingBy(
            s -> s.getDepartment(),
            Collectors.mapping(s ->s.getName(),
Collectors.toList())));

    System.out.println("\n--- Modified Report B: Department
with Names only ---");
    reportBNames.forEach((dept, names) ->
System.out.println(dept + ": " + names));
    }
}

// Explain why partitioningBy()is inappropriate for Report B?
// Ans: because it is designed to split a collection into two
groups based on a boolean predicate.
// In the case of Report B, we want to group students by their
department, which can have multiple categories (e.g., CSE, IT, ECE,
ME). ns (true and false). Instead, `groupingBy()` is the correct
choice for this scenario, as it allows us to group elements based
on a key (in this case, the department) and can handle multiple
categories effectively.

// Modify Report B so each department contains only students names?
// Ans: To modify Report B so that each department contains only
the names of the students, we can use the `Collectors.mapping()`
// collector in conjunction with `Collectors.groupingBy()`.
// This allows us to transform the list of students into a list of
their names while still grouping them by department.

```

Output:

```
PS C:\Users\ACER\Desktop\SEM 7\CSE406\CA1> cd "c:\Users\ACER\Desktop\SEM 7\CSE406\CA1\" ; if ($?) { javac Q1_Streams.java } ; if ($?) { java Q1_Streams }
--- Report A: CGPA > 8.0 ---
Score > 8.0: [Aman (8.5), Chandan (9.0), Uday (8.2)]
Score <= 8.0: [Aditya (7.5), Shivang (6.8)]

--- Report B: Group by Department ---
CSE: [Aman (8.5), Chandan (9.0)]
ECE: [Shivang (6.8)]
ME: [Uday (8.2)]
IT: [Aditya (7.5)]

--- Modified Report B: Department with Names only ---
CSE: [Aman, Chandan]
ECE: [Shivang]
ME: [Uday]
IT: [Aditya]
PS C:\Users\ACER\Desktop\SEM 7\CSE406\CA1>
```

Question 2:

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculator {
    static double num1 = 0, num2 = 0;
    static String operator = "";

    public static void main(String[] args) {
        JFrame frame = new JFrame("Simple Calculator");
        frame.setLayout(new FlowLayout());

        JTextField display = new JTextField(15);
        JLabel resultLabel = new JLabel("Result: 0");

        frame.add(display);
        frame.add(resultLabel);

        for (int i = 0; i <= 9; i++) {
            String digit = String.valueOf(i);
            JButton numBtn = new JButton(digit);
            numBtn.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                    display.setText(display.getText() + digit);
                }
            });
            frame.add(numBtn);
        }

        JButton btnPlus = new JButton("+");
        JButton btnMinus = new JButton("-");
        JButton btnMul = new JButton("*");
        JButton btnDiv = new JButton("/");
        JButton btnEqual = new JButton("=");
        JButton btnClear = new JButton("C");

        frame.add(btnPlus);
        frame.add(btnMinus);
        frame.add(btnMul);
        frame.add(btnDiv);
        frame.add(btnEqual);
        frame.add(btnClear);

        btnPlus.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (!display.getText().isEmpty()) {
                    num1 = Double.parseDouble(display.getText());
                    operator = "+";
                    display.setText("");
                    resultLabel.setText("Operator: +");
                }
            }
        });
    }
}
```

```

        btnMinus.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (!display.getText().isEmpty()) {
                    num1 = Double.parseDouble(display.getText());
                    operator = "-";
                    display.setText("");
                    resultLabel.setText("Operator: -");
                }
            }
        });

        btnMul.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (!display.getText().isEmpty()) {
                    num1 = Double.parseDouble(display.getText());
                    operator = "*";
                    display.setText("");
                    resultLabel.setText("Operator: *");
                }
            }
        });

        btnDiv.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (!display.getText().isEmpty()) {
                    num1 = Double.parseDouble(display.getText());
                    operator = "/";
                    display.setText("");
                    resultLabel.setText("Operator: /");
                }
            }
        });

        btnEqual.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (!display.getText().isEmpty() &&
!operator.isEmpty()) {
                    num2 = Double.parseDouble(display.getText());
                    double result = 0;

                    if (operator.equals("+")) result = num1 + num2;
                    else if (operator.equals("-")) result = num1 -
num2;
                    else if (operator.equals("*")) result = num1 *
num2;
                    else if (operator.equals("/")) result = (num2
!= 0) ? (num1 / num2) : 0;

                    display.setText(String.valueOf(result));
                    resultLabel.setText("Result: " + result);
                    operator = "";
                }
            }
        });

        btnClear.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {

```

```

        display.setText("");
        resultLabel.setText("Result: 0");
        num1 = 0;
        num2 = 0;
        operator = "";
    }
});

display.addKeyListener(new KeyAdapter() {
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ESCAPE) {
            display.setText("");
            resultLabel.setText("Result: 0");
            num1 = 0;
            num2 = 0;
            operator = "";
        }
    }
});

frame.addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});

btnPlus.addMouseListener(new MouseAdapter() {
    public void mouseEntered(MouseEvent e) {
        resultLabel.setText("Mouse entered + button");
    }
});

frame.setSize(220, 300);
frame.setVisible(true);
}
}

```

Output:

